



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: _____ Nicolle Escobar _____ **Carné:** _____ 1264425 _____

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
Es una interfaz de programación de aplicaciones que Utiliza el protocolo HTTP para permitir que diferentes sistemas se comuniquen entre sí de forma ligera.
2. ¿Qué es un endpoint?
Es un punto de acceso específico en un servidor donde una API puede recibir solicitudes y enviar respuestas.
3. ¿Qué diferencia existe entre una URL y un endpoint?
URL: Es la dirección global y general para localizar cualquier recurso en internet.
Endpoint: Es una ruta específica dentro de esa URL que apunta a una operación concreta de la API.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
GET: solicita o recupera información de un recurso sin modificarlo.
POST: crea un nuevo recurso en el servidor.
PUT: actualiza completamente un recurso existente (reemplaza toda su información).
PATCH: modifica de forma parcial un recurso existente (actualiza solo los campos enviados).
DELETE: elimina un recurso específico del servidor.
5. ¿Qué es JSON y por qué es común en las APIs?
Es un formato ligero de intercambio de datos basado en texto, estructurado mediante pares clave-valor y arreglos.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
Es un número de tres dígitos que el servidor devuelve en cada respuesta para informar al cliente el resultado de su solicitud.
 - 200 (OK): La solicitud se procesó con éxito y el servidor devolvió la información esperada.
 - 404 (Not Found): El servidor no encuentra el recurso o el endpoint solicitado (la ruta no existe).
 - 500 (Internal Server Error): Ocurrió un error inesperado en el servidor al intentar procesar la petición.
7. ¿Qué información se puede encontrar en la respuesta de una API?
Pueden ver indicaciones de peticiones exitosas o de fallos en los estados http, metadatos y datos reales devueltos por API.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
Permite simular el comportamiento de la API mediante servidores falsos, ayuda a diseñar y validar contratos de API y estructuras de datos antes de escribir código de backend.



9. ¿Qué es y para qué sirve Swagger UI?

Es una herramienta visual de código abierto que genera documentación interactiva para APIs basadas en la especificación OpenAPI.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Cumple con permitir a los desarrolladores hacer prototipos de ideas y verificar el comportamiento de los endpoints de manera rápida, Postman se convierte en una herramienta indispensable para las pruebas y el control de calidad.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

The first screenshot shows the Swagger UI for the GET /pet/{petId} endpoint. It includes a description 'Returns a single pet.', a parameter 'petId' of type integer, and an 'Execute' button. The second screenshot shows a curl command: 'curl -X GET "http://petstore3.swagger.io/api/v3/pet/1" -H "accept: application/json"'. The third screenshot shows the response for an invalid ID (404), with a message 'Pet not found' and a JSON body: {'code': 404, 'message': 'Pet not found'}. The response headers include 'Access-Control-Allow-Headers', 'Access-Control-Allow-Methods', 'Access-Control-Allow-Origin', 'Access-Control-Expose-Headers', 'Content-Length', 'Content-Type', 'Date', and 'Server'.

el servidor indica que no encontró una mascota registrada con el ID 1. El mensaje devuelto fue Pet not found.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags



POST /pet/{petId} Updates a pet in the store with form data.

Updates a pet resource based on the form data.

Parameters

Name	Description
petid * required integer(\$int64) (path)	ID of pet that needs to be updated
name string (query)	Name of pet that needs to be updated
status string (query)	Status of pet that needs to be updated

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet/125?name=manzana&status=available' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/125?name=manzana&status=available
```

Server response

Code **Details**

404 Error: response status is 404

Response body

```
can't parse JSON. Raw result:
Pet not found
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 13
content-type: application/json
date: Thu,01 Oct 2026 05:15:14 GMT
server: Jetty(9.4.53.v20231009)
```

Responses

Code	Description	Links
200	successful operation	No links
400	Invalid input	No links
default	Unexpected error	No links

Media type

application/json

Example Value | **Schema**

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué? PUT, porque se utilizó para actualizar la información de una mascota existente.
- ¿Qué datos fueron enviados en el body? Se enviaron los datos completos de la mascota.
- ¿Qué código HTTP devolvió la API? 200
- ¿Cómo comprobó que el cambio realmente ocurrió? Realizando posteriormente un GET /pet/{petId} con el mismo ID.

PUT /pet Update an existing pet.

Update an existing pet by Id.

Parameters

Name	Description
petid * required integer(\$int64) (path)	ID of pet to return

Execute **Clear**

Request body required

application/json

Update an existent pet in the store

Edit Value | **Schema**

```
{
  "id": 10,
  "name": "pancho modificado",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "pendig"
}
```

Responses

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet/10' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/10
```

Server response

Code **Details**

200 **Response body**

```
{
  "id": 10,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "pancho modificado",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "pendig"
}
```



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado. status
- Explique dónde se envía ese parámetro. Se envía como un query parameter, es decir, en la URL después de ?.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

The left screenshot shows the Swagger UI for the GET /pet/findByStatus endpoint. The status parameter is set to 'available'. The right screenshot shows the same endpoint with the status parameter set to 'pending'. The response body is visible, showing a pet with status 'pending'.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

The left screenshot shows a collection named 'API' with five requests: DELETE (112 B), GET (249 B), POST (231 B), PUT (239 B), and status (2.2 KB). The right screenshot shows the 'status' request with a JSON body.



7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
La API respondió con un error indicando que no se encontró el recurso.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada. GET /pet/145048
- b) Código HTTP recibido. 404
- c) Respuesta de la API. Pet not found
- d) Qué esperaba que ocurriera. Que la API indicara que no existía ninguna mascota con ese ID.
- e) Qué ocurrió realmente. La API devolvió un código 404
- f) Explicación de la causa. Se consultó un identificador que no corresponde a ninguna mascota registrada en la API.

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.